

Inhaltsverzeichnis

1. Abstraktionen in Java	1
1.1. Interfaces	1
1.1.1. Erzeugungsregeln	2
1.1.2. Warum Interfaces?	3
1.2. Abstrakte Klassen	5

1. Abstraktionen in Java

In Java bezeichnet Abstraktion das Prinzip, komplexe Sachverhalte durch **einfachere, abstraktere Modelle** darzustellen, indem unwichtige Details ausgeblendet werden. In der objektorientierten Programmierung (OOP) bedeutet dies, dass nur die relevanten Eigenschaften und Methoden einer Klasse definiert werden und die Implementierungsdetails verborgen bleiben.

Es gibt **zwei Hauptwege**, wie Abstraktion in Java umgesetzt wird, nämlich durch

1. *Interfaces* und
2. *abstrakte Klassen*

Ziele von Abstraktion

Die **Abstraktion**, als eines der wichtigsten **Kernkonzepte** in Java, wird vor allem eingesetzt, um verschiedene Erfordernisse umzusetzen. Dazu gehören u.a.

- Ermöglichen, auf einem höheren Abstraktionsniveau zu programmieren (z. B. „etwas fährt“, ohne zu kennen, *wie* genau dies erfolgt),
- Wiederverwendbarkeit und Wartbarkeit des Codes verbessern,
- Implementierungsdetails verstecken und eine klare Programmierschnittstelle für andere Entwickler bieten (Synonyme: Programmervorschrift, -vertrag, API (Application Programming Interface))

❓ In eigenen Worten: Was ist eine Programmierschnittstelle (API)? Beispiele?

1.1. Interfaces

[[Inhalt](#) | [Demo](#) | [Übungen](#)]

In Java ist ein **Interface** ein solcher abstrakter Datentyp. Dieser enthält eine Sammlung von Methoden und/oder Konstanten *ohne* Implementierung. Ein Interface dient als eine Art

Vertrag oder auch **Vorschrift**

WAS - die "Funktionalität - eine Schnittstelle zur Verfügung stellt, sie sagt aber nichts über das **WIE** aus, d.h. wie die Funktionalität umgesetzt ist.

Die Signatur des Interfaces erfordert das Schlüsselwort

```
interface
```

Eine Beispiel-Signatur

```
public interface MyInterface {  
    // Inhalt des Interfaces  
}
```

Die Implementierung eines **Interfaces** durch konkrete **Klassen** erfolgt mit dem Schlüsselwort

```
implements
```

Beispiel-Implementierungen (auch: Realisierungen)

```
public class Bmw implements Car {  
    // Implementierung des Interfaces  
}  
  
... oder ...  
  
public class Car implements Vehicle {  
    // Implementierung des Interfaces  
}
```

1.1.1. Erzeugungsregeln

In einem **Interface** ist gestattet:

- konstante Variablen
- abstrakte Methoden
- statische Methoden
- **default** Methoden

Darüber hinaus ist **wichtig**, dass ...

- Interfaces nicht direkt instanziiert werden können,
- ein Interface "*leer*" sein kann, also ohne Konstanten oder Methoden,
- alle Interface Deklarationen haben standardmäßig **public** oder **default** access, **private** Methoden (seit Java 9) sind aber auch erlaubt.

- Der **abstract** Modifizierer wird vom Compiler automatisch hinzugefügt,
- Interface Methoden nicht **protected** oder **final** sein können,
- Interface Felder (=Konstanten) per Definition **public**, **static**, und **final** sein müssen.

Die *grafische* Darstellung der Beziehung zwischen Interfaces und deren Implementierung sieht folgendermaßen aus:

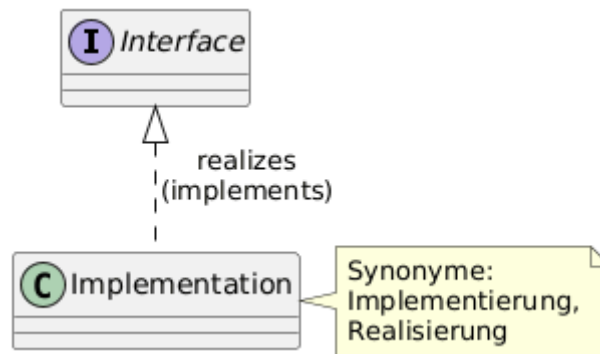


Figure 1. Interface & Realisierung

1.1.2. Warum Interfaces?

Verhaltensvorschrift

Schnittstellen werden verwendet, um bestimmte Verhaltensfunktionen zu definieren, die von "beliebigen" Klassen umgesetzt werden können. Beispiele für Java-Schnittstellen sind **Comparable**, **Comparator** und **Cloneable**. Sie werden durch konkrete Klassen implementiert.

Für mehr Information dazu siehe → [Comparable.html](#)

Mehrfach-Vererbung

Java-Klassen unterstützen nur die einfache (singuläre) Vererbung. Durch die Verwendung von Schnittstellen ist man jedoch auch in der Lage, quasi eine Mehrfachvererbungen zu implementieren.

Polymorphismus

Bei Polymorphismus handelt es sich um die Fähigkeit eines Objekts, während der Laufzeit unterschiedliche Formen anzunehmen. Genauer gesagt handelt es sich um die Nutzung der **Override**-Methode, die sich erst zur Laufzeit auf eine bestimmte Klasse bezieht.

In Java kann **Polymorphismus** auf einfache Weise mithilfe von Interfaces erreicht werden.

Beispielsweise kann eine **Shape**-Schnittstelle (s.u.) verschiedene Formen annehmen – konkret kann es ein Kreis oder ein Quadrat sein.

Am **Beispiel** der Klasse **Shape**:

```
public interface Shape {
```

```
String name();  
}
```

Die Klasse **Kreis**:

```
public class Circle implements Shape {  
  
    @Override  
    public String name() {  
        return "Circle";  
    }  
}
```

Und die **Quadrat** Klasse:

```
public class Square implements Shape {  
  
    @Override  
    public String name() {  
        return "Square";  
    }  
}
```

Und die **Nutzung** im Unit-Test sieht folgendermaßen aus:

```
@Test  
@DisplayName("Demo 2: Polymorphismus durch Interfaces")  
public void canRealizePolymorphism() {  
    // given  
    List<Shape> shapes = new ArrayList<>();  
  
    Shape circle = new Circle();  
    Shape square = new Square();  
  
    shapes.add(circle);  
    shapes.add(square);  
  
    // when  
    for (Shape shape : shapes) {  
        System.out.println(shape.name());  
    }  
  
    // then  
    assertEquals(2, shapes.size());  
}
```

Demo:

```
src/test/java/de/dhbw/demo/InterfaceDemoTest.java
```

1.2. Abstrakte Klassen

In Java gibt es auch **abstrakte Klassen**. Sie enthalten, wie ihre konkreten Pendanten, auch eine Sammlung von Attributen und Methoden, diese können nun aber *konkret oder abstrakt* sein. Sie stehen sozusagen "zwischen" konkreten Klassen und Interfaces, s.h. man kann sagen, eine abstrakte Klasse sei eine Art **Hybrid** zwischen

1. einer **konkreten Klasse** auf der einen Seite und
2. einem **abstrakten Interface** auf der anderen.

Mit einer abstrakten Klasse können die Merkmale beider Welten (konkrete & abstrakte Merkmale) umgesetzt bzw. die Vorteile beider Konzepte in *einem* bereitgestellt werden.

Schlüsselkonzepte

Das **Schlüsselwort**, der "abstrakte Modifikator", zur Umsetzung von Abstraktionen in Java ist

```
abstract
```

Die wichtigsten **Merkmale** einer abstrakten Klasse sind:

- Eine abstrakte Klasse enthält den abstrakten **Modifikator** **abstract** in der Klassensignatur
- Eine abstrakte Klasse kann von Unterklassen beerbt, aber **nicht instanziiert** werden
- Wenn eine Klasse eine oder mehrere **abstrakte Methoden** enthält, muss die Klasse selbst als abstrakt deklariert werden
- Eine abstrakte Klasse kann sowohl abstrakte als auch konkrete Methoden deklarieren
- Eine von einer abstrakten Klasse abgeleitete Unterklasse muss entweder alle abstrakten Methoden der Basisklasse realisieren bzw. implementieren oder selbst abstrakt sein

Die zugehörige **grafische Darstellung** dieser Beziehung mit Beispiel:

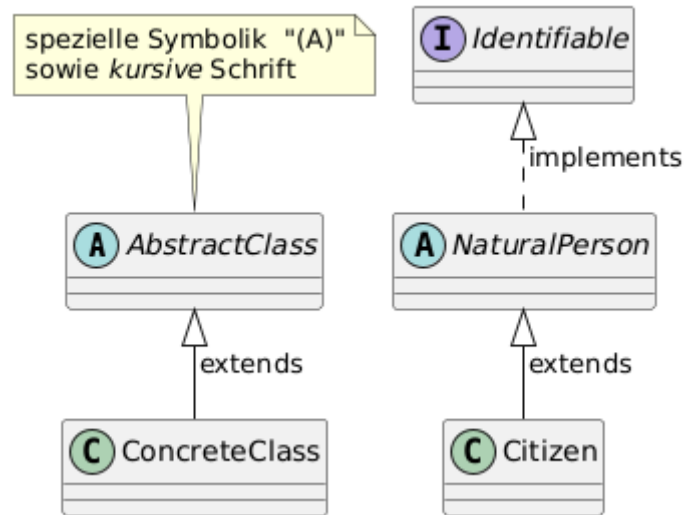


Figure 2. Abstrakte Klassen & Vererbung

Beispiel abstrakte Klasse & Methode:

```

public abstract class MyClass {
    public abstract void myMethod();
}
  
```

Demo:

→ `src/test/java/de/dhbw/demo/AbstractsDemoTest.java`

Nutzung abstrakter Klassen

Java `interface` und `abstract class` sind also beide Formen von Abstraktionen. Abstrakte Klassen werden häufig bei den folgenden Szenarien eingesetzt:

- **Kapselung** allgemeiner Funktionen in einer Klasse (**Wiederverwendung** von Code). Diese sollen von mehreren verwandten Unterklassen gemeinsam genutzt werden
- Es muss nur teilweise eine "Implementierungsvorschrift" definiert werden, die von Unterklassen leicht **erweitert** und verfeinert werden können
- Die Unterklassen müssen eine oder mehrere gemeinsame Methoden oder Felder mit **geschützten** Zugriffsmodifikatoren erben
- Da sich die Verwendung abstrakter Klassen außerdem implizit mit Basistypen und Untertypen befasst, werden außerdem die Vorteile des **Polymorphismus** genutzt

Zu beachten ist auch, dass die Wiederverwendung von Code oft ein "zwingender" Grund für die Verwendung abstrakter Klassen ist. Dazu in anderen Modulen mehr (→ Beziehungsarten zwischen Klassen)

Übungen:

Übung 1 - Interface

Erzeuge folgende **Klassen**:

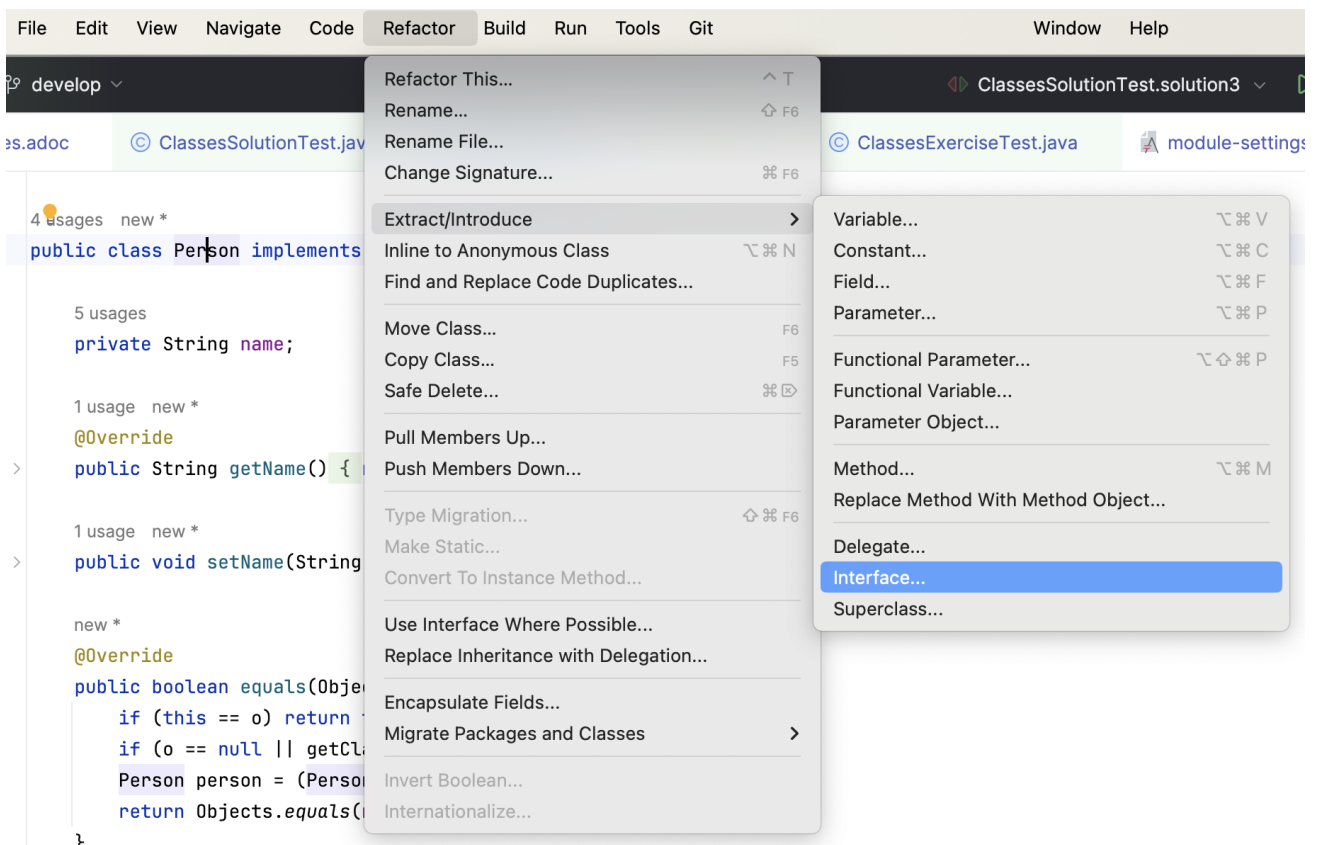
1. Ein **Interface Zug** mit den Schnittstellenmethoden
 - a. `getNumber` (soll den Wert des Feldes `number` vom Typ `String` zurückgeben) sowie
 - b. `setNumber` (soll den Wert des Feldes `number` setzen)
2. Eine konkrete Klasse `Regionalzug`, die die Schnittstelle `Zug` realisiert
3. Schreibe einen Test, setze eine Zugnummer, hole diese wieder und teste den korrekten Wert

Umsetzung in

→ `src/test/java/de/dhbw/exercise/InterfaceExerciseTest.java`

▼ Hilfe zu 'Extract Interface ...'

Cursor auf den Klassennamen, dann über das Menü ...



Übung 2 - Mini-Modell mit abstrakter Klassen und Interface

Ein Anwendungsfall (*echte Aussage aus einem Kunden-Interview*):

Es gibt zwei unterschiedliche Typen von (Zug-) Waggons. Es gibt Waggons für "Fahrgäste" und für "Frachtgüter". Als Teil eines Zuges sind die Waggons immer geordnet, sie bilden die sog. "Wagen-Reihung".

Erzeuge ein kleines, aber "vollständiges" **Klassenmodell** daraus:

1. Eine Schnittstelle **Wagon** mit Methoden
 - a. zum *Holen* und *Setzen* der Wagon-Reihenfolge (engl. **order** (Datentyp **int**))
2. Eine abstrakte Klasse **DefaultWagon**, die die Schnittstelle und die dortigen Methoden realisiert
 - a. (*Optional*) Das Feld **order** soll dabei möglichst stark geschützt sein
3. Zwei konkrete Klassen, die beide von der abstrakten Klasse *erben*, nämlich
 - a. **PassengerWagon** und
 - b. **FreightWagon**

→ `src/test/java/de/dhbw/exercise/ModelExerciseTest.java`

▼ Aufklappen mit grafischer Darstellung als kleine Hilfe...

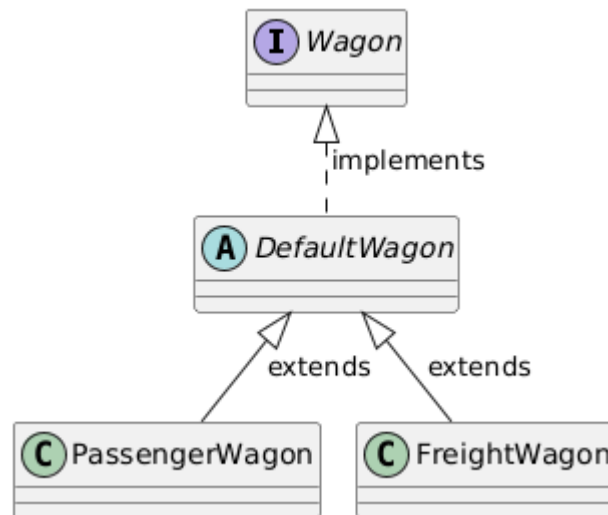


Figure 3. Ein kleines Klassenmodell